

CampOps Modular Architecture Guide

Campground Management Platform - Migration to Modular Monolith

Version: 1.0

Last Updated: November 4, 2025

Platform: CampOps - Multi-tenant Campground Management SaaS

Current Stack: Next.js 15, Express.js, Supabase (PostgreSQL), Stripe

Table of Contents

1. [Current State Assessment](#)
2. [Proposed Module Architecture](#)
3. [Migration Strategy](#)
4. [Module Definitions](#)
5. [API Reorganization](#)
6. [Database Schema Evolution](#)
7. [Implementation Roadmap](#)

Current State Assessment

Your Existing Architecture

Technology Stack:

- **Frontend:** Next.js 15 (App Router), React 19, TypeScript, TailwindCSS, Shadcn/UI
- **Backend:** Express.js API server
- **Database:** Supabase (PostgreSQL with RLS)
- **Payments:** Stripe (Connect + Subscriptions)
- **Testing:** Vitest, Playwright, Testing Library

Current Folder Structure:

```
oh-saas/campsite-command-center/  
├── src/                # Next.js frontend  
│   ├── app/           # App Router pages  
│   ├── components/    # React components  
│   ├── lib/           # Utilities and business logic  
│   ├── hooks/         # Custom React hooks  
│   └── types/         # TypeScript type definitions  
└── api/               # Express.js API server
```

```
├── database/          # Supabase migrations
├── tests/             # Test suites
│   ├── unit/
│   ├── integration/
│   ├── security/
│   └── e2e/
└── scripts/          # Build utilities
```

Current Database Schema Analysis

Based on your actual schema, here's what you have:

Core Tables (Already Multi-tenant)

1. **companies** - Your SaaS tenants

- id (PK)
- name, owner_id
- stripe_customer_id, subscription_id
- subscription_status, subscription_plan
- billing_cycle
- onboarding_token (for invites)

2. **properties** - Campgrounds (tenant isolation via company_id)

- id (PK), company_id (FK to companies)
- name, slug, subdomain
- location data (address, city, state, zip)
- property_type (campground, rv_park, glamping, mixed)
- amenities (JSONB)
- check_in/check_out times, timezone
- stripe_account_id (Connect)
- onboarding wizard tracking
- booking page configuration
- renewal_settings (JSONB)

3. **sites** - Campsites within properties

- id (PK), property_id (FK)
- site_number, site_name, site_type
- max_occupancy, max_vehicles, size_sqft
- hookups, amenities (JSONB)
- status (available, reserved, booked, occupied, maintenance)
- pricing (base_price, weekend_price, seasonal_pricing JSONB)
- accessibility features
- pet policies

4. **guests** - Camper profiles

- id (PK), property_id (FK), user_id (FK)
- name, email, phone
- address details
- emergency contact
- stripe_customer_id

5. reservations - Bookings

- id (PK), property_id, site_id, guest_id
- confirmation_number (unique)
- check_in/check_out dates
- occupancy (adults, children, pets, vehicles)
- booking_type (nightly, weekly, monthly, seasonal, long_term)
- status (pending, confirmed, checked_in, checked_out, cancelled)
- payment_status (unpaid, partial, paid, refunded)
- extension tracking (is_extension_of, times_extended)
- renewal tracking (renewal_status, renewal_offered_at)
- check-in/check-out tracking

6. payments - Payment records

- id (PK), property_id, reservation_id
- amount, payment_method, payment_status
- stripe_payment_id, transaction_id

7. payment_installments - Payment plans

- id (PK), reservation_id
- installment_number, amount_cents
- due_date, status
- reminder tracking

8. reservation_actions - Audit trail

- id (PK), reservation_id
- action_type (created, extended, renewed, modified, etc.)
- action_details (JSONB)
- performed_by, price_change_cents
- previous_state, new_state (JSONB)

9. property_staff - RBAC

- id (PK), property_id, user_id
- role (owner, manager, staff, viewer)
- permissions (JSONB)

What You Have vs. What's Needed

✓ Already Implemented:

- Multi-tenant architecture (companies → properties)
- Row-level security policies (Supabase RLS)
- Comprehensive booking system with extensions/renewals
- Payment processing with Stripe
- Payment installment plans
- Audit trails (reservation_actions)
- Staff management with RBAC
- Onboarding wizards
- Custom booking pages

▢ Needs Refactoring:

- Business logic scattered across frontend and API
- No clear module boundaries
- Tight coupling between features
- Limited separation of concerns

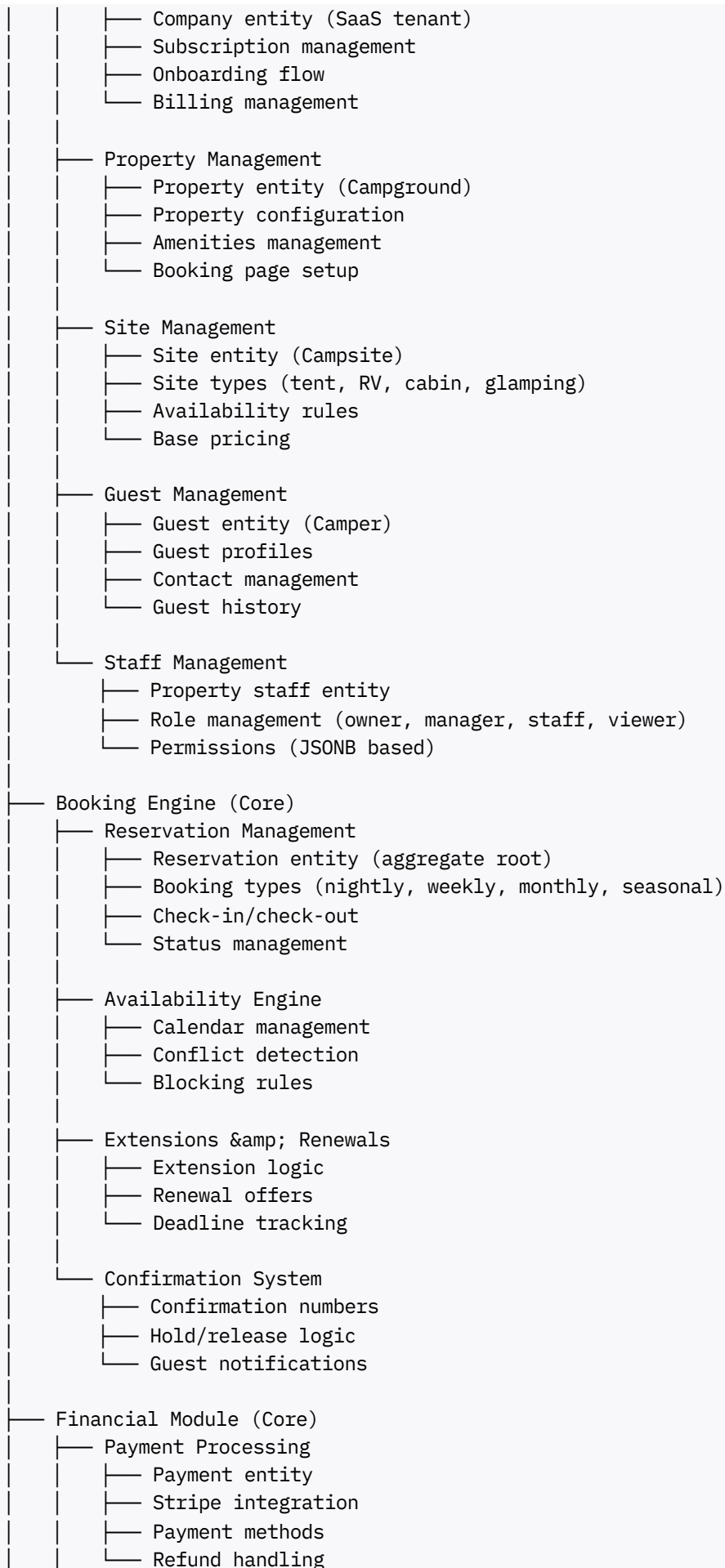
▢ Premium Modules to Add:

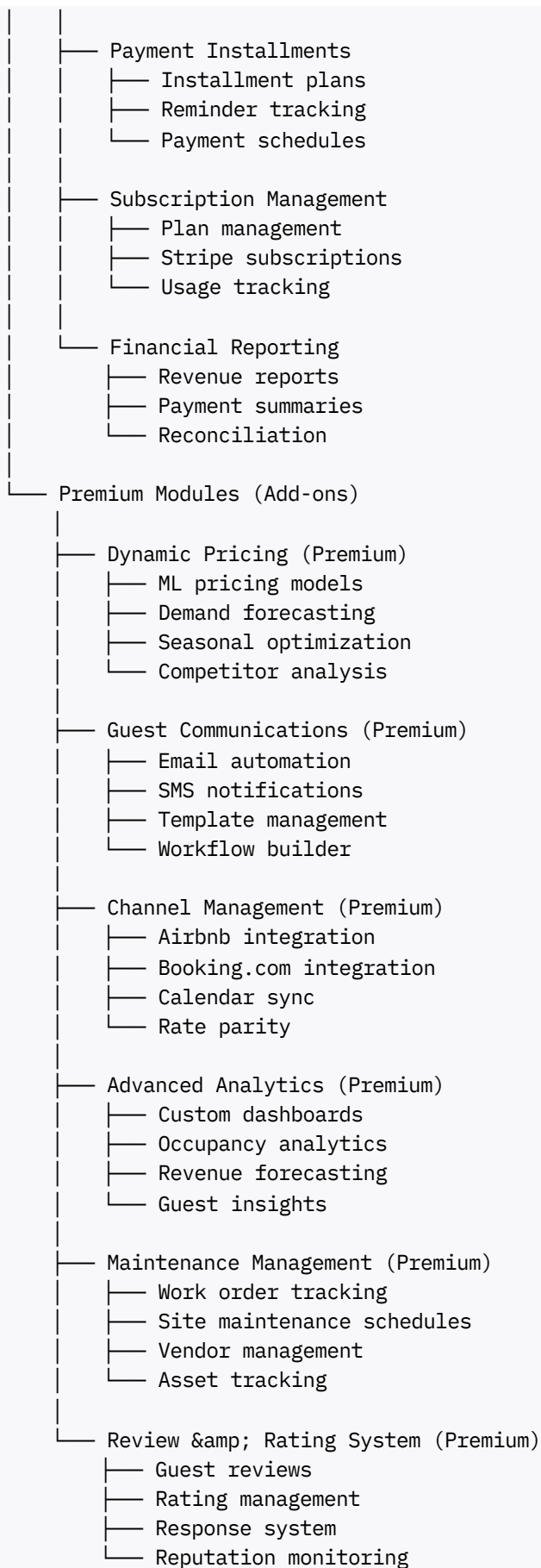
- Dynamic pricing engine (AI-powered)
- Guest communication automation
- Channel management (Airbnb, [Booking.com](#))
- Advanced analytics & reporting
- Maintenance management
- Review & rating system

Proposed Module Architecture

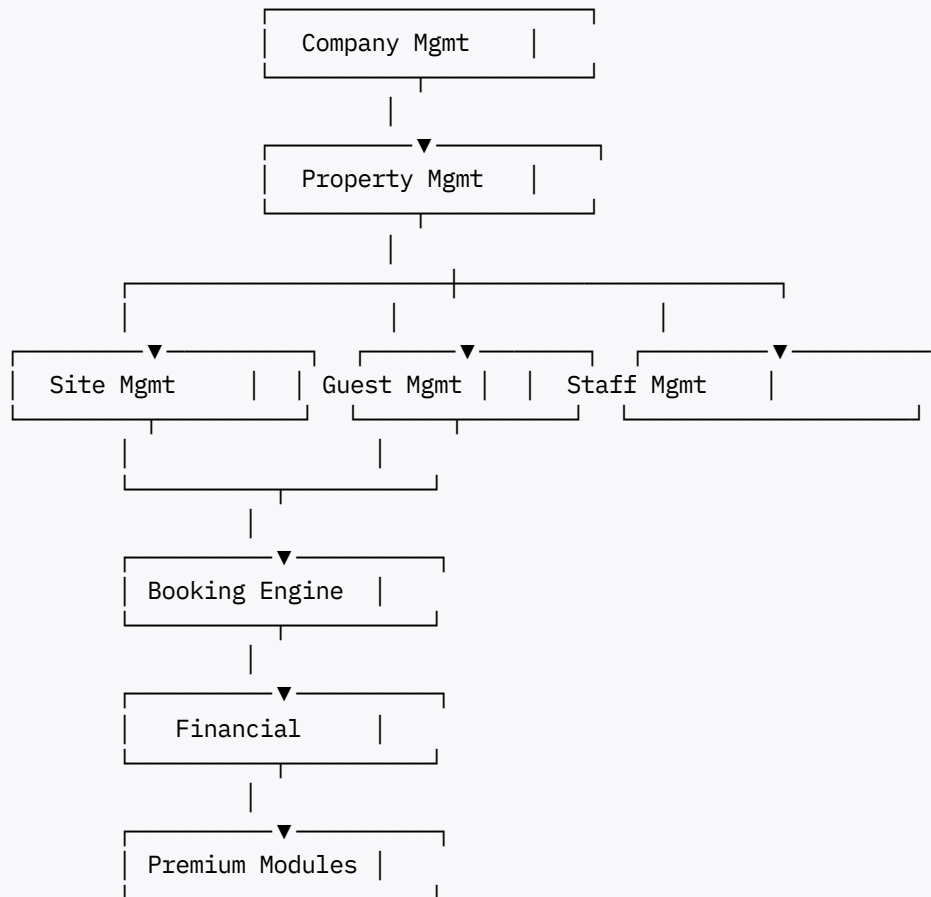
CampOps Module Structure

```
CampOps Platform
|
├── Shared Kernel
|   ├── Domain (Entity, AggregateRoot, ValueObject)
|   ├── Infrastructure (Database, EventBus, Logging)
|   └── Auth (Multi-tenant context, RLS helpers)
|
├── Core Modules (Always Active)
|   └── Company Management
```





Module Relationships



Migration Strategy

Phase 1: Shared Kernel Setup (Week 1-2)

Goal: Create foundation for modular architecture

Tasks:

1. Create `src/shared/` directory structure
2. Implement base domain classes
3. Set up event bus (in-memory initially)
4. Create database context helpers

New Structure:

```
src/shared/
├── domain/
│   ├── Entity.ts
│   ├── AggregateRoot.ts
│   └── ValueObject.ts
```

```

├── DomainEvent.ts
├── IEventBus.ts
├── infrastructure/
│   ├── database/
│   │   ├── SupabaseContext.ts
│   │   └── TenantContext.ts
│   ├── eventBus/
│   │   └── InMemoryEventBus.ts
│   ├── logging/
│   │   └── Logger.ts
├── types/
│   └── Common.ts

```

Example: Base Entity:

```

// src/shared/domain/Entity.ts
export abstract class Entity<T> {
  protected readonly _id: T;
  protected readonly _createdAt: Date;
  protected _updatedAt: Date;

  constructor(id: T, createdAt?: Date) {
    this._id = id;
    this._createdAt = createdAt || new Date();
    this._updatedAt = new Date();
  }

  get id(): T { return this._id; }
  get createdAt(): Date { return this._createdAt; }
  get updatedAt(): Date { return this._updatedAt; }

  equals(entity: Entity<T>): boolean {
    return this._id === entity._id;
  }
}

```

Phase 2: First Module - Site Management (Week 3-4)

Why Site Management First?

- Relatively independent
- Clear bounded context
- No complex dependencies
- Good learning module

New Structure:

```

src/modules/
├── SiteManagement/
│   ├── domain/
│   │   ├── entities/
│   │   └── Site.ts

```



```

├── SiteType.ts
├── valueObjects/
│   ├── Pricing.ts
│   ├── Hookup.ts
│   └── Amenity.ts
├── repositories/
│   └── ISiteRepository.ts
├── events/
│   ├── SiteCreatedEvent.ts
│   └── SiteUpdatedEvent.ts
├── application/
│   ├── commands/
│   │   ├── CreateSite/
│   │   └── UpdateSite/
│   └── queries/
│       ├── GetSite/
│       └── ListSites/
├── infrastructure/
│   └── persistence/
│       └── SupabaseSiteRepository.ts
└── api/
    ├── controllers/
    │   └── SiteController.ts
    ├── dtos/
    │   └── SiteDTO.ts
    └── routes.ts

```

Migration Steps:

1. Extract Site entity from current code:

```

// Current: scattered logic
// New: src/modules/SiteManagement/domain/entities/Site.ts

export class Site extends AggregateRoot<string> {
  private constructor(
    id: string,
    private _propertyId: string,
    private _siteNumber: string,
    private _siteName: string | null,
    private _siteType: SiteType,
    private _maxOccupancy: number,
    private _pricing: Pricing,
    private _amenities: Amenity[],
    private _status: SiteStatus
  ) {
    super(id);
  }

  static create(params: CreateSiteParams): Site {
    const site = new Site(
      uuidv4(),
      params.propertyId,
      params.siteNumber,
      params.siteName,

```

```

        params.siteType,
        params.maxOccupancy,
        params.pricing,
        params.amenities,
        SiteStatus.AVAILABLE
    );

    site.addDomainEvent(
        new SiteCreatedEvent(
            params.companyId,
            site.id,
            site.propertyId,
            site.siteNumber
        )
    );

    return site;
}

// Business logic methods
markAsReserved(): void {
    if (this._status === SiteStatus.MAINTENANCE) {
        throw new Error('Cannot reserve site under maintenance');
    }
    this._status = SiteStatus.RESERVED;
}

// Getters
get propertyId(): string { return this._propertyId; }
get siteNumber(): string { return this._siteNumber; }
get pricing(): Pricing { return this._pricing; }
// ...
}

```

2. Create repository implementation:

```

// src/modules/SiteManagement/infrastructure/persistence/SupabaseSiteRepository.ts

export class SupabaseSiteRepository implements ISiteRepository {
    constructor(private supabase: SupabaseClient) {}

    async save(site: Site): Promise<void> {
        const { error } = await this.supabase
            .from('sites')
            .insert({
                id: site.id,
                property_id: site.propertyId,
                site_number: site.siteNumber,
                site_name: site.siteName,
                site_type: site.siteType,
                base_price: site.pricing.basePrice,
                // ... map all fields
            });

        if (error) throw new Error(`Failed to save site: ${error.message}`);
    }
}

```

```

    }

    async findById(id: string): Promise<Site | null> {
        const { data, error } = await this.supabase
            .from('sites')
            .select('*')
            .eq('id', id)
            .single();

        if (error || !data) return null;

        return this.mapToEntity(data);
    }

    private mapToEntity(row: any): Site {
        return Site.reconstitute({
            id: row.id,
            propertyId: row.property_id,
            siteNumber: row.site_number,
            // ... map all fields
        });
    }
}

```

3. Create API controller:

```

// src/modules/SiteManagement/api/controllers/SiteController.ts

export class SiteController {
    constructor(
        private createSiteHandler: CreateSiteHandler,
        private getSiteHandler: GetSiteHandler
    ) {}

    async createSite(req: Request, res: Response): Promise<void> {
        const companyId = req.user.companyId; // From auth middleware

        const command = new CreateSiteCommand(
            companyId,
            req.body.propertyId,
            req.body.siteNumber,
            req.body.siteType,
            req.body.maxOccupancy,
            req.body.basePrice
        );

        const result = await this.createSiteHandler.execute(command);

        res.status(201).json({
            success: true,
            data: result
        });
    }
}

```

Phase 3: Remaining Core Modules (Week 5-12)

Week 5-6: Property Management

- Extract Property entity
- Implement property configuration
- Migrate property settings

Week 7-8: Guest Management

- Extract Guest entity
- Implement guest profiles
- Migrate contact management

Week 9-10: Booking Engine

- Extract Reservation aggregate
- Implement availability engine
- Migrate extension/renewal logic

Week 11-12: Financial Module

- Extract Payment entity
- Implement installment logic
- Migrate Stripe integration

Phase 4: Premium Modules (Week 13-20)

Week 13-15: Dynamic Pricing

- ML model integration
- Demand forecasting
- Seasonal optimization

Week 16-17: Guest Communications

- Email/SMS automation
- Template system
- Workflow builder

Week 18-20: Channel Management

- Airbnb API integration
- [Booking.com](https://www.booking.com) integration
- Calendar synchronization

Module Definitions

Company Management Module

Bounded Context: SaaS tenant organization and subscription management

Entities:

- Company (Aggregate Root)
- Subscription
- OnboardingToken

Responsibilities:

- Company creation and configuration
- Subscription lifecycle (Stripe integration)
- Billing cycle management
- Onboarding token generation
- Company settings

Current Tables:

- companies
- subscription_events

API Endpoints:

```
POST    /v1/companies
GET     /v1/companies/:id
PATCH  /v1/companies/:id
POST    /v1/companies/:id/subscription
GET     /v1/companies/:id/subscription
POST    /v1/companies/:id/invite
```

Events Published:

- CompanyCreatedEvent
- SubscriptionActivatedEvent
- SubscriptionCancelledEvent
- InviteGeneratedEvent

Property Management Module

Bounded Context: Campground configuration and management

Entities:

- Property (Aggregate Root)
- PropertySettings
- BookingPageConfiguration

Value Objects:

- Address
- OperatingHours
- CheckInPolicy

Responsibilities:

- Campground creation and configuration
- Amenity management
- Booking page setup
- Stripe Connect integration
- Wizard progress tracking
- Renewal settings

Current Tables:

- properties

API Endpoints:

```
POST /v1/properties
GET /v1/properties/:id
PATCH /v1/properties/:id
GET /v1/properties/:id/settings
PATCH /v1/properties/:id/settings
POST /v1/properties/:id/connect-stripe
```

Events Published:

- PropertyCreatedEvent
- PropertyUpdatedEvent
- StripeConnectedEvent
- OnboardingCompletedEvent

Site Management Module

Bounded Context: Campsite inventory and availability

Entities:

- Site (Aggregate Root)

Value Objects:

- Pricing (base, weekend, seasonal)
- Hookup
- Amenity
- Coordinates

Responsibilities:

- Site creation and configuration
- Site type management (tent, RV, cabin, glamping)
- Pricing configuration
- Amenity tracking
- Status management
- Pet policies
- Accessibility features

Current Tables:

- sites

API Endpoints:

```
POST   /v1/properties/:propertyId/sites
GET    /v1/properties/:propertyId/sites
GET    /v1/properties/:propertyId/sites/:id
PATCH /v1/properties/:propertyId/sites/:id
DELETE /v1/properties/:propertyId/sites/:id
GET    /v1/properties/:propertyId/sites/available
```

Events Published:

- SiteCreatedEvent
- SiteUpdatedEvent
- SiteStatusChangedEvent
- SitePricingUpdatedEvent

Guest Management Module

Bounded Context: Camper profiles and contact management

Entities:

- Guest (Aggregate Root)

Value Objects:

- ContactInfo
- Address
- EmergencyContact

Responsibilities:

- Guest profile creation
- Contact information management
- Emergency contact tracking
- Guest history
- Stripe customer creation

Current Tables:

- guests

API Endpoints:

```
POST   /v1/guests
GET     /v1/guests/:id
PATCH  /v1/guests/:id
GET     /v1/guests?propertyId=:propertyId
GET     /v1/guests/:id/reservations
```

Events Published:

- GuestCreatedEvent
- GuestUpdatedEvent
- StripeCustomerCreatedEvent

Booking Engine Module

Bounded Context: Reservation lifecycle management

Entities:

- Reservation (Aggregate Root)
- ReservationExtension

- `RenewalOffer`

Value Objects:

- `DateRange`
- `Occupancy` (adults, children, pets, vehicles)
- `VehicleInfo`

Responsibilities:

- Reservation creation (all booking types)
- Availability checking
- Conflict detection
- Check-in/check-out processing
- Extensions and modifications
- Renewal management
- Status transitions
- Hold/release logic

Current Tables:

- `reservations`
- `reservation_actions` (audit trail)

API Endpoints:

```
POST   /v1/reservations
GET    /v1/reservations/:id
PATCH /v1/reservations/:id
POST   /v1/reservations/:id/extend
POST   /v1/reservations/:id/renew
POST   /v1/reservations/:id/check-in
POST   /v1/reservations/:id/check-out
POST   /v1/reservations/:id/cancel
GET    /v1/properties/:propertyId/availability
```

Events Published:

- `ReservationCreatedEvent`
- `ReservationConfirmedEvent`
- `ReservationExtendedEvent`
- `ReservationRenewedEvent`
- `GuestCheckedInEvent`
- `GuestCheckedOutEvent`
- `ReservationCancelledEvent`

Events Subscribed:

- `PaymentCompletedEvent` → Confirm reservation
- `PaymentFailedEvent` → Release hold

Financial Module

Bounded Context: Payment processing and financial tracking

Entities:

- `Payment` (Aggregate Root)
- `PaymentInstallment`
- `Subscription`

Value Objects:

- `Money`

Responsibilities:

- Payment processing (Stripe)
- Payment installment management
- Refund processing
- Payment plan scheduling
- Reminder tracking
- Financial reporting

Current Tables:

- `payments`
- `payment_installments`
- `subscription_events`

API Endpoints:

```
POST  /v1/payments
GET    /v1/payments/:id
POST   /v1/payments/:id/refund
GET    /v1/reservations/:id/payments
POST   /v1/reservations/:id/installments
GET    /v1/reservations/:id/installments
PATCH /v1/installments/:id/mark-paid
GET    /v1/properties/:propertyId/financial-summary
```

Events Published:

- `PaymentProcessedEvent`

- PaymentCompletedEvent
- PaymentFailedEvent
- RefundProcessedEvent
- InstallmentCreatedEvent
- InstallmentPaidEvent
- InstallmentOverdueEvent

Events Subscribed:

- ReservationCreatedEvent → Set up payment schedule
- ReservationExtendedEvent → Add additional charges

API Reorganization

Current API Structure (Express)

```
api/
├── index.ts           # Main server
├── routes/
│   ├── properties.ts
│   ├── sites.ts
│   ├── reservations.ts
│   ├── guests.ts
│   └── payments.ts
├── middleware/
│   ├── auth.ts
│   └── tenant.ts
├── utils/
│   └── supabase.ts
```

Proposed Modular API Structure

```
api/
├── index.ts           # Main server
├── shared/
│   ├── middleware/
│   │   ├── auth.ts
│   │   ├── tenant.ts
│   │   └── errorHandler.ts
│   ├── utils/
│   │   └── supabase.ts
├── modules/
│   ├── CompanyManagement/
│   │   ├── controllers/
│   │   │   └── CompanyController.ts
│   │   ├── routes.ts
│   │   └── index.ts
├── PropertyManagement/
```

```

├── controllers/
│   └── PropertyController.ts
├── routes.ts
├── index.ts
├── SiteManagement/
│   ├── controllers/
│   │   └── SiteController.ts
│   ├── routes.ts
│   └── index.ts
├── GuestManagement/
│   ├── controllers/
│   │   └── GuestController.ts
│   ├── routes.ts
│   └── index.ts
├── BookingEngine/
│   ├── controllers/
│   │   ├── ReservationController.ts
│   │   └── AvailabilityController.ts
│   ├── routes.ts
│   └── index.ts
├── Financial/
│   ├── controllers/
│   │   ├── PaymentController.ts
│   │   └── InstallmentController.ts
│   ├── routes.ts
│   └── index.ts
└── server.ts

```

Migration Example: Reservations API

Before (Current):

```

// api/routes/reservations.ts
router.post('/reservations', async (req, res) => {
  // Business logic mixed with API logic
  const { propertyId, siteId, guestId, checkIn, checkOut } = req.body;

  // Direct Supabase call
  const { data, error } = await supabase
    .from('reservations')
    .insert({ ... });

  res.json(data);
});

```

After (Modular):

```

// api/modules/BookingEngine/controllers/ReservationController.ts
export class ReservationController {
  constructor(
    private createReservationHandler: CreateReservationHandler,
    private eventBus: IEventBus
  ) {}
}

```

```

async create(req: Request, res: Response): Promise<void> {
  const companyId = req.user.companyId;

  const command = new CreateReservationCommand(
    companyId,
    req.body.propertyId,
    req.body.siteId,
    req.body.guestId,
    req.body.checkInDate,
    req.body.checkOutDate,
    req.body.occupancy
  );

  const result = await this.createReservationHandler.execute(command);

  res.status(201).json({
    success: true,
    data: result
  });
}
}

```

Database Schema Evolution

Current Schema → Modular Schema Mapping

Your current schema is already well-structured! Here's how it maps to modules:

Company Management:

- `companies` ✓ (already has everything needed)
- `subscription_events` ✓

Property Management:

- `properties` ✓ (comprehensive)

Site Management:

- `sites` ✓ (well-structured)

Guest Management:

- `guests` ✓

Booking Engine:

- `reservations` ✓ (excellent with extension/renewal tracking)
- `reservation_actions` ✓ (audit trail)

Financial:

- `payments` ✓

- `payment_installments` ✓ (great for payment plans)

Staff Management:

- `property_staff` ✓

Recommended Schema Enhancements

1. Add `event_store` table for domain events:

```
CREATE TABLE IF NOT EXISTS public.event_store (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_id UUID NOT NULL UNIQUE,
  event_type VARCHAR(255) NOT NULL,
  aggregate_id UUID NOT NULL,
  aggregate_type VARCHAR(100) NOT NULL,
  company_id UUID NOT NULL REFERENCES public.companies(id),
  data JSONB NOT NULL,
  metadata JSONB DEFAULT '{}',
  occurred_at TIMESTAMP WITH TIME ZONE NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT now()
);

CREATE INDEX idx_event_store_event_type ON public.event_store(event_type);
CREATE INDEX idx_event_store_aggregate ON public.event_store(aggregate_id, aggregate_type);
CREATE INDEX idx_event_store_company ON public.event_store(company_id);
```

2. Add `module_licenses` table for premium features:

```
CREATE TABLE IF NOT EXISTS public.module_licenses (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES public.companies(id),
  module_name VARCHAR(100) NOT NULL,
  enabled BOOLEAN DEFAULT false,
  trial_ends_at TIMESTAMP WITH TIME ZONE,
  subscription_starts_at TIMESTAMP WITH TIME ZONE,
  subscription_ends_at TIMESTAMP WITH TIME ZONE,
  settings JSONB DEFAULT '{}',
  created_at TIMESTAMP WITH TIME ZONE DEFAULT now(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT now(),

  UNIQUE(company_id, module_name),
  CHECK (module_name IN (
    'core',
    'dynamic_pricing',
    'guest_communications',
    'channel_management',
    'advanced_analytics',
    'maintenance_management',
    'review_system'
  ))
);
```

Implementation Roadmap

Weeks 1-2: Foundation

- ☐ Set up shared kernel
- ☐ Create base domain classes
- ☐ Implement in-memory event bus
- ☐ Add event_store table
- ☐ Add module_licenses table

Weeks 3-4: Site Management Module

- ☐ Extract Site entity
- ☐ Create repository implementation
- ☐ Build command/query handlers
- ☐ Set up API controller
- ☐ Write tests

Weeks 5-6: Property Management Module

- ☐ Extract Property entity
- ☐ Migrate property configuration
- ☐ Implement settings management
- ☐ Update API endpoints

Weeks 7-8: Guest Management Module

- ☐ Extract Guest entity
- ☐ Implement contact management
- ☐ Migrate Stripe customer logic

Weeks 9-10: Booking Engine Module

- ☐ Extract Reservation aggregate
- ☐ Implement availability engine
- ☐ Migrate extension/renewal logic
- ☐ Build check-in/check-out flows

Weeks 11-12: Financial Module

- ☐ Extract Payment entity
- ☐ Implement installment logic
- ☐ Migrate Stripe integration

Weeks 13+: Premium Modules

- ☐ Dynamic pricing engine
- ☐ Guest communications
- ☐ Channel management
- ☐ Advanced analytics

Next Steps

1. **Review this document** with your team
2. **Prioritize modules** based on business needs
3. **Start with shared kernel** (foundation)
4. **Extract one module** (Site Management recommended)
5. **Iterate and refine** patterns

Your existing schema is excellent - the migration will primarily focus on organizing business logic into modules while keeping your database structure largely intact.

Would you like detailed implementation guides for specific modules?